

同济大学
TONGJI UNIVERSITY

人工智能应用

目　录

同济大学 TONGJI UNIVERSITY

一、自然语言中机器翻译

二、图像分类与视觉对象定位

三、语音识别与合成

四、机器人技术

五、智能家居与物联网

六、金融行业

七、医疗健康

自然语言中机器翻译

自然语言中机器翻译

同济大学 TONGJI UNIVERSITY

机器翻译任务

定义：机器翻译（Machine Translation, MT）是指利用计算机自动将一种自然语言（源语言）的文本转换为另一种自然语言（目标语言）的文本的过程，旨在保持语义一致性的同时，生成符合目标语言语法和表达习惯的译文。

- **传统机器翻译方法：**
  - 基于文法的机器翻译（Grammar-based Machine Translation）
  - 基于统计的机器翻译（Statistical Machine Translation, SMT）
- **现代机器翻译方法：**
  - 基于深度学习的神经机器翻译（Neural Machine Translation, NMT）

基于文法的机器翻译

定义：通过语言学和语法规则进行源语言与目标语言之间的单词和结构转换。这种方法的核心在于使用语言的语法规则进行精确的语言转换。

主要步骤：

1. **解析（Parsing）：**
    1. 将源语言句子解析成语法结构树或其他中间表示。
    2. 语法解析器根据源语言的语法规则生成句子的结构树，这个结构树能表达出句子各个部分之间的关系。
2. **生成（Generation）：**
    1. 根据目标语言的语法规则，利用解析得到的中间表示生成目标语言句子。
    2. 这个过程依赖于目标语言的语法和句法规则来保证句子的自然流畅。

基于文法的机器翻译-应用案例

- **法律文件翻译：**法律文本有着严格的语言结构和术语要求，因此基于文法的机器翻译常用于法律领域。在翻译时，系统可以使用文法规则来确保句法结构的正确性。例如，利用抽象句法树（Abstract Syntax Tree, AST）转换可以确保源语言的法律术语和结构准确转换为目标语言的法律术语，并维持法律文书的逻辑性。

- **医学文献翻译：**医学文献有着复杂的句法结构和专有术语，基于文法的机器翻译可以通过准确的语法规则来处理这些挑战。在此类应用中，系统不仅要保证医学术语的准确翻译，还要确保句子的结构符合目标语言的医学写作规范。

- **技术文档翻译：**工程或技术领域的文档通常有着特定的句法模式，文法驱动的翻译系统可以通过语言规则准确地将技术文档从一种语言翻译成另一种语言。例如，工程设计手册或产品使用说明书的翻译，基于文法的翻译方法能确保准确传达技术内容。

自然语言中机器翻译

同济大学 TONGJI UNIVERSITY

基于文法的机器翻译- python实现

```python
# 基于文法的机器翻译
def grammar_based_translation(source_sentence, grammar_rules):
    # 对源语言句子进行词法和句法分析
    syntax_tree = parse_sentence_with_grammar(source_sentence, grammar_rules)

    # 生成目标语言句子的结构
    target_sentence_structure = translate_syntax_tree(syntax_tree, grammar_rules)

    # 将目标语言的句法结构转换为完整的句子
    target_sentence = generate_sentence_from_structure(target_sentence_structure)

    return target_sentence

# 例子
source_sentence = "I love programming."
grammar_rules = load_grammar_rules("english_to_chinese")
translated_sentence = grammar_based_translation(source_sentence, grammar_rules)
print(translated_sentence)  # 输出翻译后的句子
```

基于统计的机器翻译

定义：
这种方法不依赖于语言学规则，而是通过大规模的双语语料库来学习源语言和目标语言之间的翻译概率。**核心思想**为通过对源语言和目标语言句子的对齐，统计源语言单词与目标语言单词之间的转换概率，从而实现自动翻译。

主要步骤：

1. **训练阶段：**
    1. **双语语料库构建**：收集大规模的双语语料库，其中包括源语言和目标语言一一对齐的句子对。
    2. **学习翻译模型**：通过对这些对齐句子对的学习，获得源语言单词和目标语言单词之间的翻译概率。
    3. **统计建模**：使用统计方法（如隐马尔可夫模型、n-gram模型）来建立源语言与目标语言之间的翻译概率模型。
2. **解码阶段：**
    1. **翻译解码**：在解码阶段，对于给定的源语言句子，利用训练好的翻译模型，计算源语言句子与目标语言句子之间的翻译概率。
    2. **选择最优翻译**：采用贪婪搜索（Greedy Search）或束搜索（Beam Search）等算法，找到具有最高概率的目标语言句子。

基于统计的机器翻译-应用案例

- **Google 翻译**：Google翻译初期就采用了基于统计的机器翻译方法，使用大规模的双语语料库通过词汇对齐、短语对齐、词法映射等技术来进行翻译。SMT通过在大量平行语料上训练翻译模型，能够有效提高翻译的流畅度和准确性，尤其在语种之间有足够大语料库的情况下。

- **国际商务文件翻译**：在国际商务和合同文件翻译中，SMT可以通过统计模型处理大量的合同、报告、发票等文档。该方法通过对大规模语料的统计学习，能够提高对常用短语、术语的翻译精度，减少手动翻译的时间和成本。

- **政府和公共服务领域翻译**：例如，许多政府机构和公共服务系统使用基于统计的机器翻译技术来快速翻译多种语言的文件、报告和公告。在这种应用中，SMT能够有效处理多语言翻译需求，尤其在多语种的文档翻译中表现突出。

自然语言中机器翻译

同济大学 TONGJI UNIVERSITY

基于统计的机器翻译- python实现

```python
# 基于统计的机器翻译
def statistical_translation(source_sentence, phrase_table, language_model):
    # 将源语言句子分解为短语单元
    source_phrases = segment_sentence_into_phrases(source_sentence)

    # 使用短语对齐表找到最优的目标语言短语
    target_phrases = []
    for phrase in source_phrases:
        best_match = get_best_translation(phrase, phrase_table)
        target_phrases.append(best_match)

    # 使用语言模型对目标语言短语进行排序和优化
    target_sentence = generate_target_sentence_from_phrases(target_phrases, language_model)

    return target_sentence

# 例子
source_sentence = "I love programming."
phrase_table = load_phrase_table("english_to_chinese")
language_model = load_language_model("chinese_model")
translated_sentence = statistical_translation(source_sentence, phrase_table, language_model)
print(translated_sentence)  # 输出翻译后的句子
```

自然语言中机器翻译

同济大学 TONGJI UNIVERSITY

基于深度学习的机器翻译

基于深度学习的机器翻译（Neural Machine Translation，NMT）是一种通过深度神经网络模型实现的翻译方法。该方法通过在大规模对齐的双语语料库上进行端到端训练，能够直接从源语言输入生成目标语言输出。NMT能够从大量的双语数据中学习语言的潜在特征和语义表达，从而更好地捕捉语言之间复杂的关系和长距离的依赖。

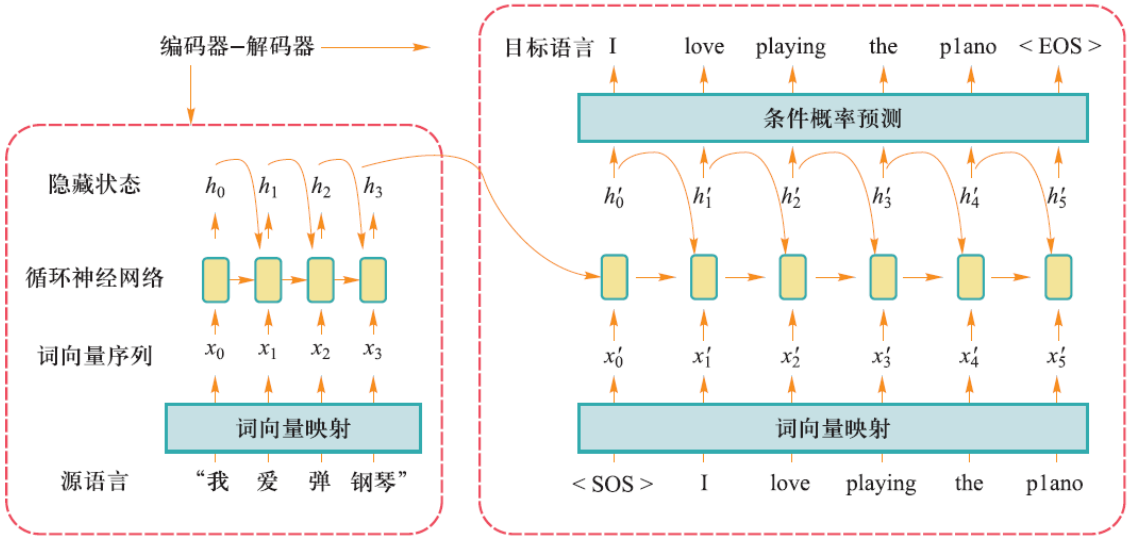


图 10.10 基于编码器 – 解码器模型的机器翻译示意

自然语言中机器翻译

同济大学 TONGJI UNIVERSITY

基于深度学习的机器翻译

编码器-解码器（Encoder-Decoder）模型是NMT中的一种常见架构。该架构的编码器将源语言的文本转换为词向量表示，而解码器则根据编码器的输出生成目标语言的翻译。这种端到端的结构使得模型能够学习源语言和目标语言单词之间的对应关系，从而提高翻译的准确性和质量。

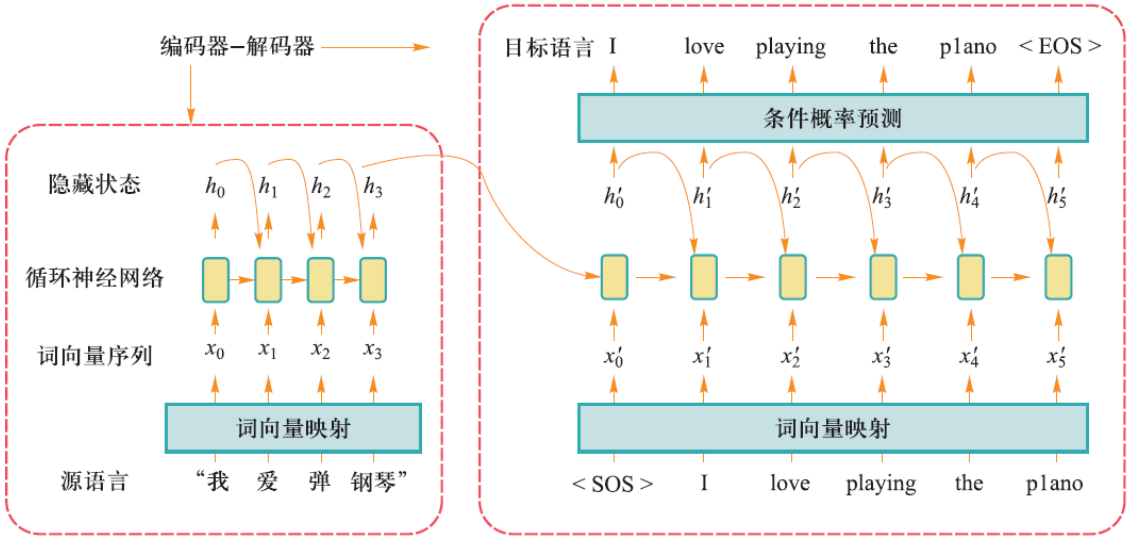


图 10.10 基于编码器 – 解码器模型的机器翻译示意

自然语言中机器翻译

同济大学 TONGJI UNIVERSITY

基于深度学习的机器翻译

编码器

编码器的作用是将源语言句子中的每个单词及其上下文信息转化为固定维度的向量表示。具体来说，编码器在每个时间步骤上，通过当前输入单词的向量和前一时刻的隐藏状态来生成当前单词的隐藏状态。这个隐藏状态代表了当前单词以及其上下文的综合信息。

编码器逐一处理源语言中的每个单词，直到完成整个句子的处理。当所有单词处理完毕后，编码器的最后一个隐藏状态就成为了源语言句子的表示。这个隐藏状态将作为解码器的初始输入，启动解码过程，生成目标语言中的第一个单词。

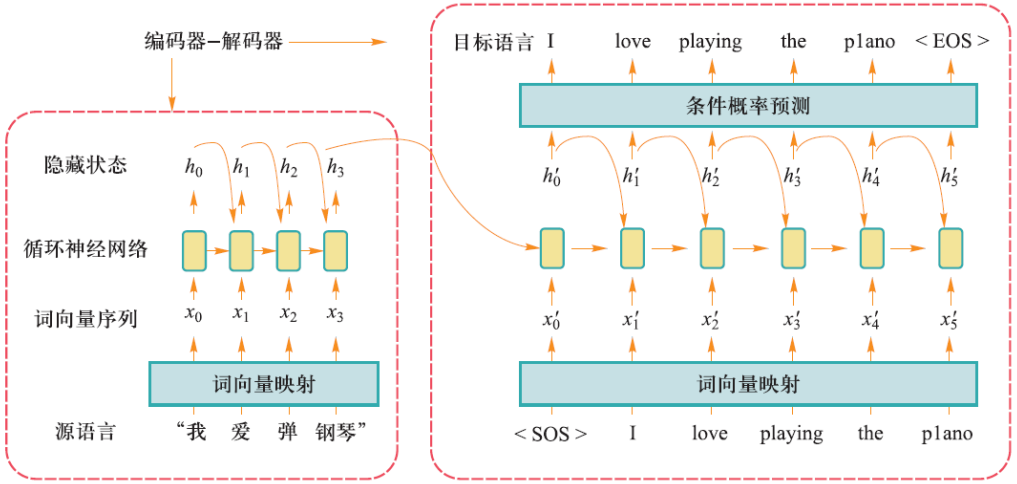


图 10.10 基于编码器 – 解码器模型的机器翻译示意

自然语言中机器翻译

同济大学 TONGJI UNIVERSITY

基于深度学习的机器翻译

解码器

解码器将编码器在处理完源语言句子后生成的隐藏状态作为输入，逐步生成目标语言中的每个单词。在每个时间步骤，解码器依赖于前一个时刻的隐藏状态和前一时刻生成的输出单词（即前序词）来计算当前时刻的隐藏状态。接着，解码器根据当前隐藏状态计算条件概率，生成目标语言词表中每个候选单词的概率分布，并选择概率最大的单词作为当前时刻的输出单词。

在生成目标语言的第一个单词时，解码器通常会接收到一个开始标记（如 ）作为第一个输出单词的前序词。为了控制解码器的生成过程，通常会引入一个特殊的终止符号（例如 ），表示翻译过程的结束。当解码器生成终止符号时，源语言句子“我爱弹钢琴”就被翻译为“I love playing the piano”，整个翻译过程也随之完成。

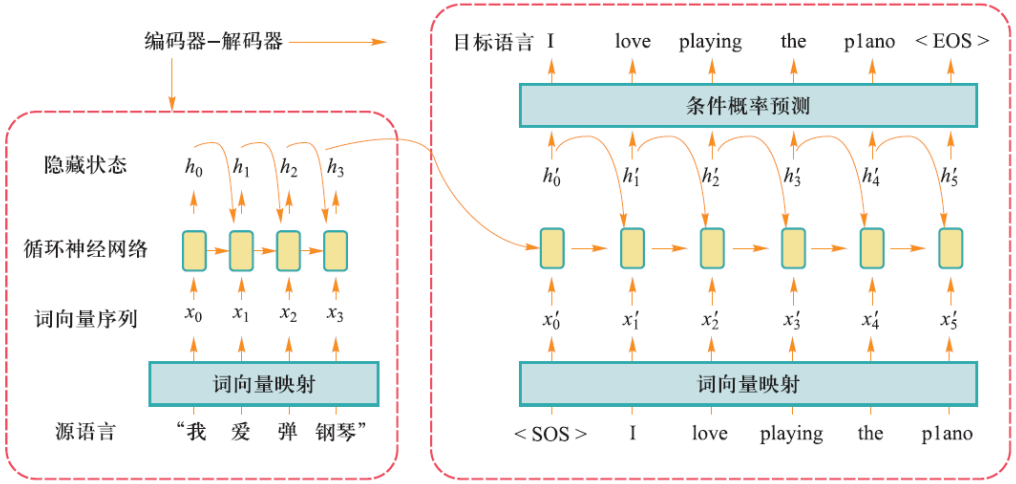


图 10.10 基于编码器 – 解码器模型的机器翻译示意

基于深度学习的机器翻译

模型训练

训练编码器-解码器模型需要准备对齐的源语言和目标语言语料。在训练过程中，解码器生成的单词与目标语言中的真实单词进行对比，使用交叉熵损失函数衡量生成译文与真实译文之间的差异。根据计算出的损失，通过误差反向传播算法优化模型参数，常用优化方法如随机梯度下降（SGD）或Adam。通过多次迭代，模型逐步学习源语言和目标语言之间的映射关系，提升翻译质量。

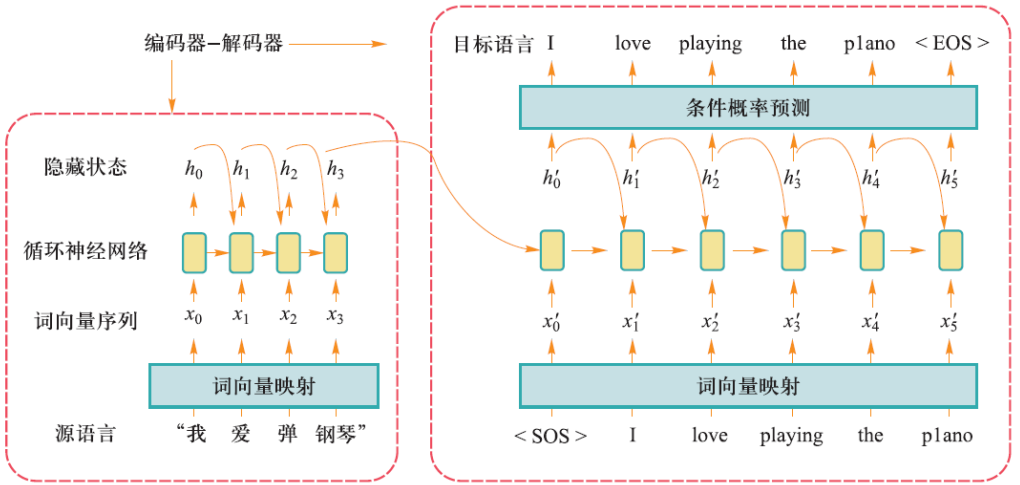


图 10.10 基于编码器 – 解码器模型的机器翻译示意

基于深度学习的机器翻译-应用案例

- **DeepL 翻译：**DeepL翻译系统采用了神经网络模型，使用大型数据集进行训练。DeepL的神经机器翻译能够提供比传统机器翻译更自然、更流畅的翻译，尤其在复杂句子结构、词汇选择和上下文理解方面具有明显优势。DeepL被认为在很多语言对的翻译中优于传统的SMT和基于规则的翻译系统。

- **社交媒体内容翻译：**社交媒体平台，如Facebook和Twitter，利用神经机器翻译系统来实时翻译用户生成的内容，跨语言连接全球用户。NMT通过对用户生成内容（如帖子、评论等）的处理，能够提供高质量的实时翻译，尽量保持语气、情感和上下文的一致性。

- **多语言客服系统：**许多大型跨国公司采用NMT技术来支持多语言客服聊天机器人。通过神经网络模型，这些系统可以处理来自全球用户的查询，并提供高质量的自动翻译回应。NMT能够有效理解客户提出的复杂问题，并根据上下文生成自然流畅的回答，改善客户服务体验。

自然语言中机器翻译

同济大学 TONGJI UNIVERSITY

基于深度学习的机器翻译- python实现

```python
# 基于深度学习的神经机器翻译
def neural_machine_translation(source_sentence, encoder_model, decoder_model):
    # 对源语言句子进行编码，获取上下文向量
    encoder_output = encoder_model.encode(source_sentence)

    # 解码上下文向量生成目标语言句子
    target_sentence = decoder_model.decode(encoder_output)

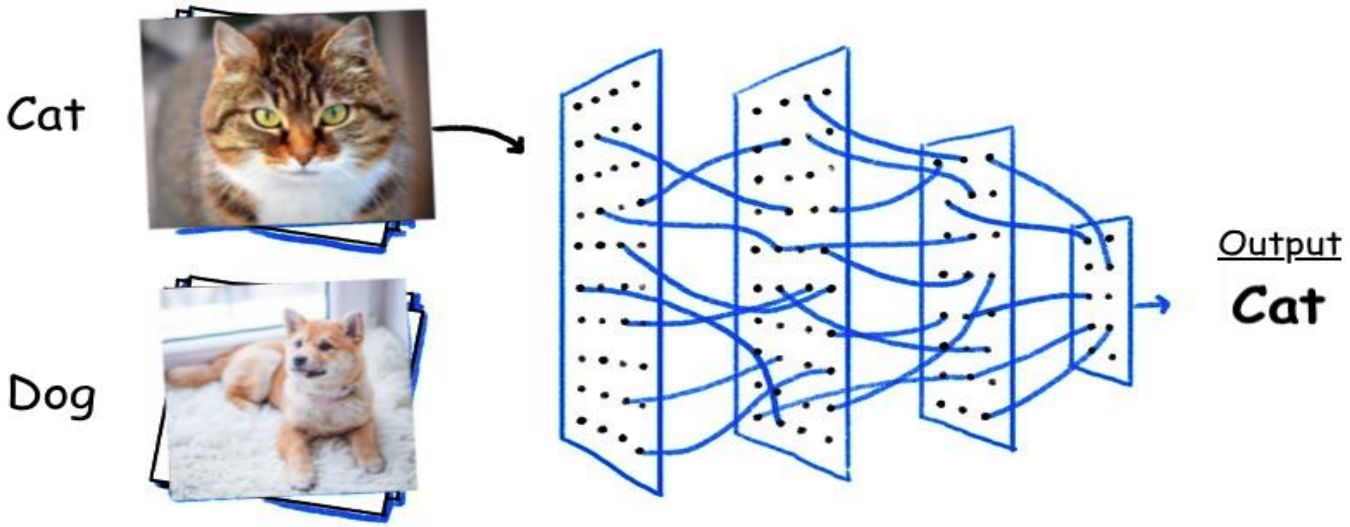
    return target_sentence


# 例子
source_sentence = "I love programming."
encoder_model = load_encoder_model("english_encoder")
decoder_model = load_decoder_model("chinese_decoder")
translated_sentence = neural_machine_translation(source_sentence, encoder_model, decoder_model)
print(translated_sentence)  # 输出翻译后的句子
```

图像分类与视觉对象定位

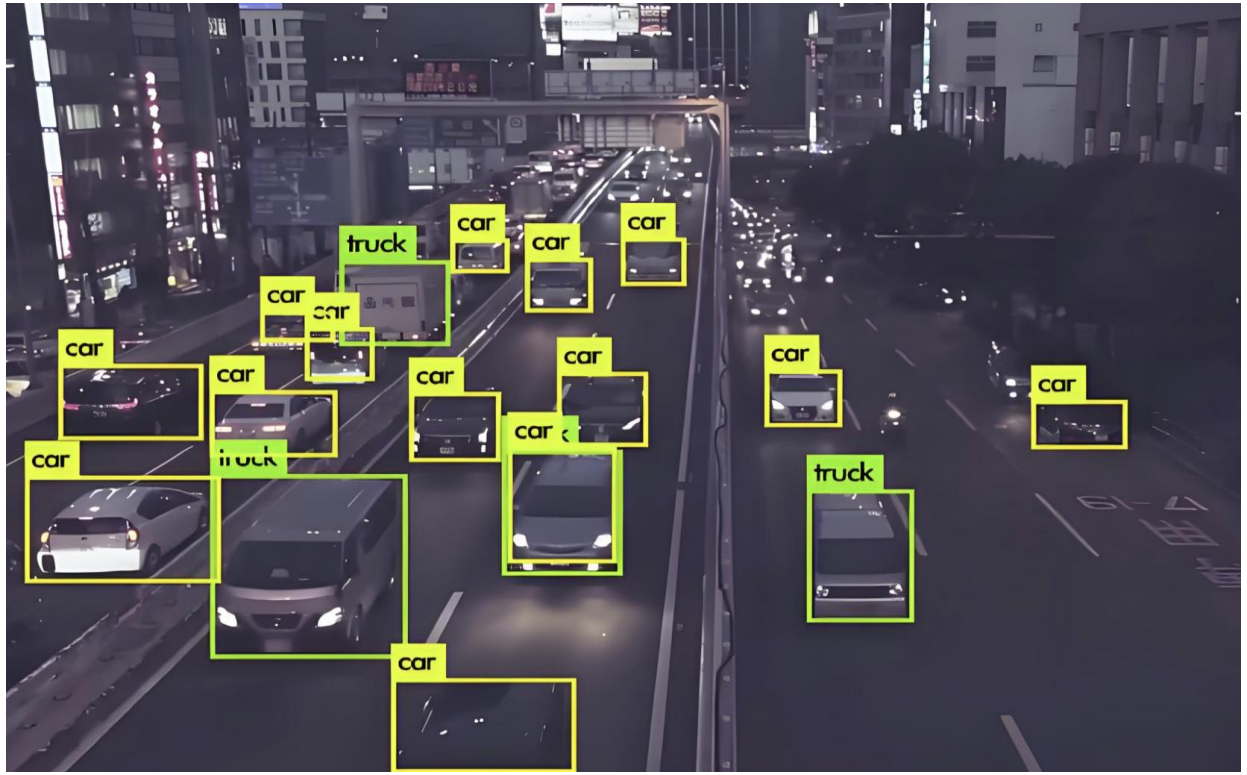
图像分类

图像分类任务指的是人工智能模型将输入的图像识别并归属于K个类别中的某一类别。模型的目标是通过分析图像中的特征，确定图像属于哪个预定义类别。例如，识别图片中的猫、狗或汽车等。



视觉对象定位

视觉对象定位是指人工智能模型不仅识别图像中的目标物体，还能够输出这些目标物体的位置信息。这通常通过框定物体的位置（如通过边界框的坐标）来实现，用于精确标定图像中的多个物体。



图像分类与视觉对象定位

同济大学 TONGJI UNIVERSITY

卷积神经网络

卷积神经网络（CNN）是解决图像分类和视觉对象定位问题的典型深度学习技术。CNN通过多个卷积层进行逐层抽象和特征提取，每一层通过卷积操作提取图像的不同特征，从而逐步构建对目标物体的深刻理解。

- **CNN架构**：该结构通常由多个卷积层、池化层以及全连接层组成，逐渐提取更复杂的图像特征，最后输出预测结果。
- **多任务学习**：图10.11展示了一个卷积神经网络采用多任务学习的示例，模型同时进行图像分类和视觉对象定位任务。通过共享卷积层的特征提取部分，模型能够在处理图像分类的同时，准确预测图像中物体的位置信息。

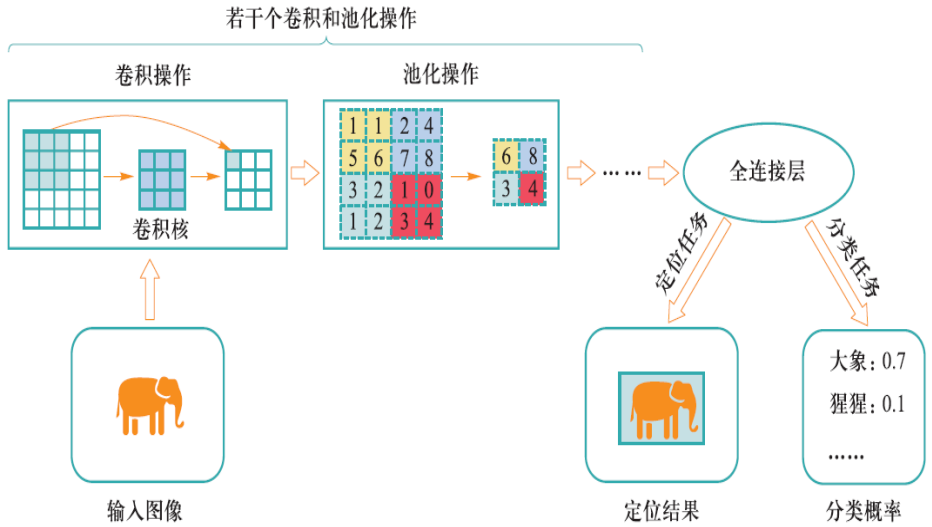


图 10.11　基于卷积神经网络的图像分类和视觉对象定位示意

卷积神经网络

- **卷积操作：**图像是高度非结构化的信息，卷积操作用于从图像中提取关键特征，通过卷积核扫描图像，捕捉局部区域的模式。
- **池化操作：**池化操作用于降维和提取图像中的统计特征，常见的池化方法有最大池化和平均池化。最大池化选择区域中的最大值，而平均池化则计算区域内的平均值，帮助提高模型的鲁棒性。
- **全连接层：**全连接层用于执行线性变换和非线性变换。线性变换通过权重矩阵和偏置处理输入特征向量，输出新的特征向量；激活函数对输出进行非线性投影，增强模型的非线性表达能力。

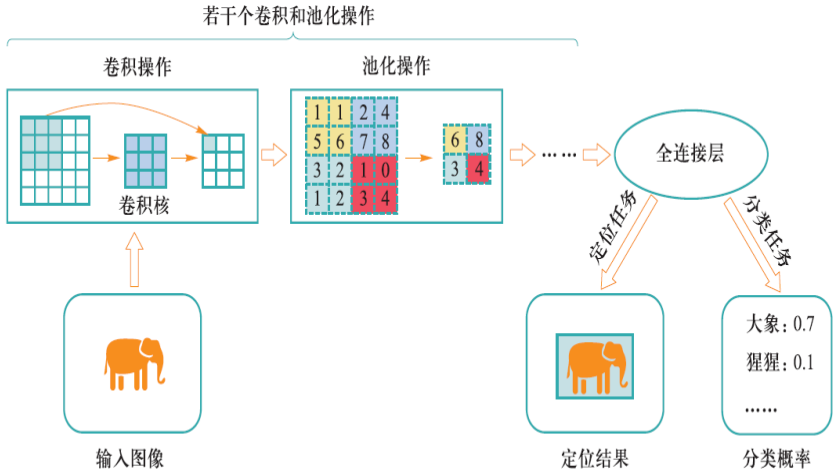


图 10.11　基于卷积神经网络的图像分类和视觉对象定位示意

图像分类与视觉对象定位

同济大学 TONGJI UNIVERSITY

卷积神经网络

- **分类任务：**全连接层输出的特征向量通过Softmax激活函数进行分类。Softmax函数将特征向量映射为各类别的概率分布，选择概率最大的类别作为图像分类的结果。
- **视觉物体定位任务：**在视觉物体定位任务中，经过全连接层的特征向量被用来预测物体的位置信息。参考Faster RCNN模型，利用**区域候选网络（RPN）生成多个候选区域，并通过感兴趣区域池化层（RoI Pooling）**提取这些区域的特征，最终确定每个候选区域的物体类别和位置信息。RoI池化将不同大小的候选区域转换为固定大小的特征矩阵，确保后续处理的一致性。

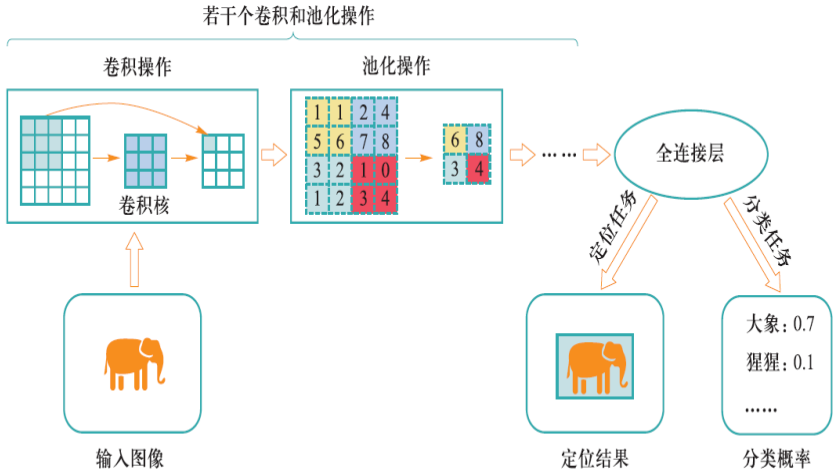


图 10.11　基于卷积神经网络的图像分类和视觉对象定位示意

应用案例

自动驾驶

- 图像分类：自动驾驶系统需要能够对道路上不同的物体进行分类，如行人、交通标志、其他车辆等，以便做出相应的反应。
- 视觉对象定位：同时，自动驾驶系统还需要定位图像中的物体（例如，识别道路上的行人并确定其位置），以确保车辆的安全行驶。

安防监控

- 图像分类：监控摄像头可以实时监测不同类型的物体（如人、车辆、包裹等），对可疑活动进行报警。
- 视觉对象定位：通过对象定位，系统可以确定图像中物体的确切位置，并在监控图像上框出这些物体以便追踪。

医学影像分析

- 图像分类：医学影像分析系统可以通过分类来识别不同类型的病变区域，如肿瘤、结节等。
- 视觉对象定位：定位系统可以在影像中标注出病变区域的位置，帮助医生进行进一步的分析和诊断。

图像分类与视觉对象定位

同济大学 TONGJI UNIVERSITY

图像分类–python实现

```python
# 图像分类
import tensorflow as tf
from tensorflow.keras.preprocessing import image
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input, decode_predictions

# 加载预训练模型
model = MobileNetV2(weights='imagenet')

# 加载图像并预处理
img_path = 'path_to_image.jpg'
img = image.load_img(img_path, target_size=(224, 224))
img_array = image.img_to_array(img)
img_array = tf.expand_dims(img_array, axis=0)  # 添加批次维度
img_array = preprocess_input(img_array)

# 图像分类预测
predictions = model.predict(img_array)

# 解码并显示预测结果
decoded_predictions = decode_predictions(predictions, top=3)[0]
for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
    print(f"{i+1}: {label} ({score:.2f})")
```

图像分类与视觉对象定位

同济大学 TONGJI UNIVERSITY

视觉对象定位-python实现

```python
# 视觉对象定位
import cv2
import numpy as np
import tensorflow as tf
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.preprocessing import image
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input

model = tf.keras.applications.MobileNetV2(weights='imagenet')       # 加载预训练的对象检测模型

img_path = 'path_to_image.jpg'               # 加载图片并进行预处理
img = cv2.imread(img_path)
height, width, _ = img.shape

img_resized = cv2.resize(img, (224, 224))            # 预处理图像
img_array = np.array(img_resized)
img_array = np.expand_dims(img_array, axis=0)
img_array = preprocess_input(img_array)

predictions = model.predict(img_array)           # 使用模型进行预测

startX, startY, endX, endY = 50, 50, 200, 200         # 假设的边界框坐标

cv2.rectangle(img, (startX, startY), (endX, endY), (0, 255, 0), 2)           # 绘制边界框
cv2.putText(img, 'Object', (startX, startY - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.9, (0, 255, 0), 2)

cv2.imshow('Object Localization', img)           # 显示图像
cv2.waitKey(0)
cv2.destroyAllWindows()
```

语音识别与合成

基于隐马尔科夫链的语音识别

1970年前后，美国数学家 Baum等人建立起了马尔可夫链（Markov Chain）的数学基础，后来有学者将隐马尔科夫链模型（Hidden Markov Model，HMM）应用于语音识别任务，实现了从单个单词语音识别到连续单词语音识别的飞跃。

该方法通过马尔科夫过程来刻画模拟语音发声机理，每一个语音序列相当于状态转移图中一条路径，在这一模型中，语音识别任务就是从所有可能的语句序列中选择出最可能状态转移序列。模型通过EM算法进行迭代优化训练，预测时则使用被称为维特比算法（Viterbi algorithm）完成解码。

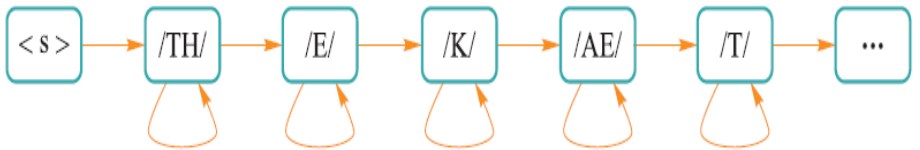


图 10.12　语句“The cat…”的状态转移过程

基于神经网络模型的语音识别

语音识别任务可用机器翻译任务中常用的序列-序列（sequence to sequence）框架完成。然而语音识别任务中输入和输出之间具有单调对齐性质，即输入语音和输出文本单词在次序上需保持完全一致，而不像翻译等任务中会出现不同语种间单词在表达相同语义时会发生单词次序调换现象。为了充分利用这一性质，研究者提出了**基于连接时序分类**（Connectionist Temporal Classification，CTC）的语音模型训练方法 [Graves2014]。

在传统语音识别模型中，训练数据中语音片段与文字序列需要按照单词级别、甚至是音素级别进行对齐，才能进行有效训练。但是，基于神经网络的语音识别方法只要进行句子级别文本序列和音频片段对齐，完成端到端的模型训练。为此，需要将语音识别任务中原本不定长的序列-序列问题转化为完全对齐的帧到帧问题。

基于神经网络模型的语音识别

为了实现这一目的，连接时序分类首先在输出序列所有符号中不同位置增加一个空白符ϵ，对输出序列进行扩充，从而使得扩充后输出序列与输入音频特征序列具有相同长度。例如，对于短句$the\ cat$，下面两条增加了若干空白符ϵ的序列都是对原有序列的合理扩充：

$$t,\epsilon,\epsilon,h,\epsilon,e,\epsilon,\epsilon,\backslash b,\epsilon,\epsilon,c,\epsilon,\epsilon,a,\epsilon,\epsilon,\epsilon,t,\epsilon$$
$$t,h,\epsilon,\epsilon,\epsilon,e,\epsilon,\epsilon,\epsilon,\backslash b,c,\epsilon,\epsilon,\epsilon,a,\epsilon,\epsilon,\epsilon,t,\epsilon$$

最终，模型通过最大化所有这些合理序列的输出概率之和来进行训练。

CTC方法聚焦于一个输入序列（如语音序列）到一个输出序列（如单词序列）的结果，只关心模型预测输出序列是否与真实序列接近（相同），而不用关心所预测输出序列中每个结果在某些时间点上是否与输入的序列正好对齐。

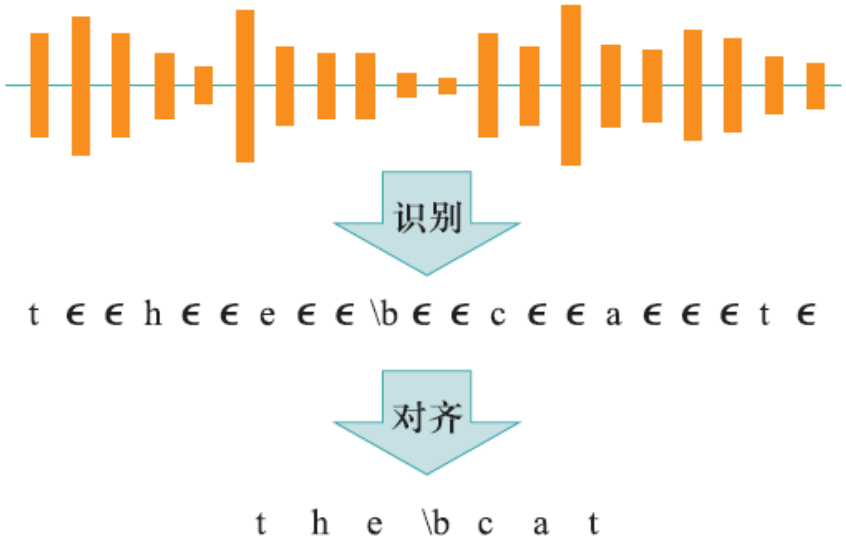


图 10.13　连接时序分类模型的语音识别过程

语音合成

语音合成（speech synthesis）是一种通过机器合成人声的技术。将文本转化为声学特征的系统被称为声学模型（acoustic model），将声学特征输入转化为音频输出的系统则称为声码器（vocoder），完整的从文本到语音的转换系统又被称为TTS（text-to-speech）系统。

- 传统基于序列对序列（seq2seq）的深度学习模型在进行合成时必须按序以自回归方式逐帧生成合成音频，这使得合成实时性大打折扣。为了解决这一问题，全并行非自回归架构合成模型FastSpeech[Ren 2019]被提出。FastSpeech利用了文本和语音单调对应性质，通过长度适配模块预测并控制语音速度和韵律，可并行完成语音合成。在图10.14中，短句*the cat*中每个字母根据“3,2,3,3,3,4,2“要求被扩充，然后对扩充后的序列进行处理，合成得到对应语音结果。在图中可见输入序列每帧均与输出序列每帧一一对应，因此可对整个序列进行并行处理，而不像seq2seq任务的输出需要自回归地依序预测。
- 上述方法需要用到音频短时帧（音素）与文本字符级别的对齐信息，导致人工标注成本非常高。为了解决这一问题，FastSpeech使用预训练的基于Transformer的自回归语音合成模型，通过自回归模型中文本和语音序列的互注意力矩阵信息，自动生成音素和字符级别之间的对齐标注信息，用于非自回归模型的训练。

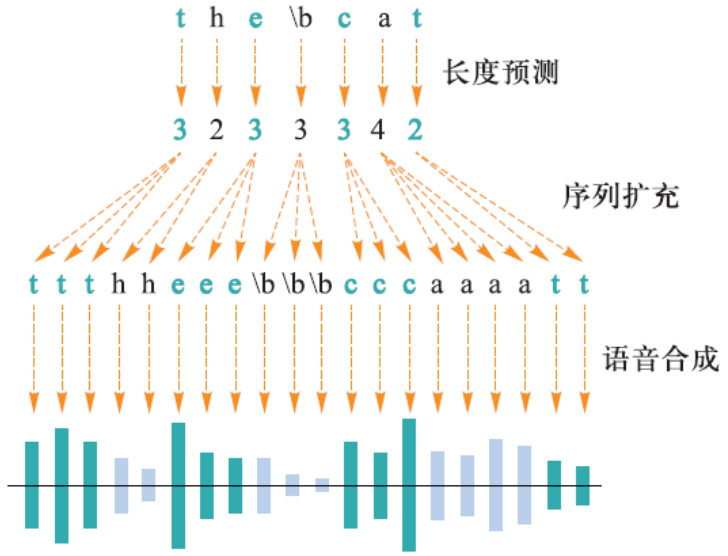


图 10.14　非自回归模式下的语音合成

语音识别与合成

同济大学 TONGJI UNIVERSITY

应用案例

智能助手（如 Siri, Google Assistant, Alexa）

- 语音识别：用户通过语音指令与智能助手互动，例如“播放音乐”或“设置闹钟”。
- 语音合成：智能助手在响应时，通过语音合成技术将文字转化为语音，给用户反馈。

无障碍服务（语音助手为视障人士提供帮助）

- 语音识别：帮助视障人士通过语音输入文字信息或控制设备，例如“发送短信”或“查询天气”。
- 语音合成：将界面内容和反馈信息通过语音朗读出来，帮助视障用户获取信息。

智能客服

- 语音识别：客服系统能够识别客户的语音问题，例如“我想查询我的账户余额”。
- 语音合成：客服系统可以通过语音回答客户的问题，例如“您的账户余额是500元”。

语音翻译

- 语音识别：将用户的语音输入转化为文本。
- 语音合成：通过翻译系统将文本翻译为目标语言，并通过语音合成技术发音给用户。

语音识别与合成

语音识别- python实现

```python
import speech_recognition as sr

# 初始化语音识别器
recognizer = sr.Recognizer()

# 使用麦克风输入
with sr.Microphone() as source:
    print("请说话...")
    recognizer.adjust_for_ambient_noise(source)  # 调整噪声
    audio = recognizer.listen(source)  # 监听并录制语音

# 识别语音内容
try:
    text = recognizer.recognize_google(audio, language="zh-CN")  # 使用Google语音识别API
    print("识别的内容是: ", text)
except sr.UnknownValueError:
    print("未能识别语音")
except sr.RequestError as e:
    print(f"无法连接到语音识别服务; {e}")
```

语音识别与合成

同济大学 TONGJI UNIVERSITY

语音合成- python实现

```python
import pyttsx3

# 初始化语音合成引擎
engine = pyttsx3.init()

# 设置语音合成的属性（如语速、音量等）
engine.setProperty('rate', 150)  # 设置语速
engine.setProperty('volume', 1)  # 设置音量，范围为0到1

# 语音合成并播放
text_to_speech = "您好，欢迎使用语音合成服务！"
engine.say(text_to_speech)

# 等待语音播放完毕
engine.runAndWait()
```

机器人技术

机器人控制

机器人通过多种传感器（Sensor）得到对环境（Environment）的感知，进而得到对智能体和环境的状态估计（State Estimation），根据具体任务和规划算法实现基本的规划（Planning），进而调动低阶控制器控制机器人的自身机械结构、操作单元（Manipulator）执行相应动作，最后通过一系列高阶控制器（Controller）实现整体状态控制直到完成任务



图 10.18　机器人完成任务的基本流程

应用案例-工业机器人

- 应用场景：自动化生产线、焊接、装配、搬运等
- 案例：工业机器人用于汽车制造业中的焊接和装配，能极大提高生产效率，减少人工劳动强度。它们通过精准的操作来执行重复性的工作，保证产品的一致性和高精度。
- 技术要求：包括机械臂运动控制、路径规划、力控制、感知系统等。

```python
import numpy as np
# 简单的2D网格地图，0表示空地，1表示障碍物
map = np.array([[0, 0, 1, 0, 0],
                [0, 1, 1, 0, 0],
                [0, 0, 0, 1, 0],
                [0, 1, 0, 0, 0],
                [0, 0, 0, 0, 0]])
# 机器人起点和终点
start = (0, 0)
goal = (4, 4)
# 计算路径的函数
def find_path(start, goal, map):
    open_list = [start]
    closed_list = set()
    parent_map = {start: None}

    while open_list:
        current = open_list.pop(0)
        # 如果到达终点，返回路径
        if current == goal:
            path = []
            while current:
                path.append(current)
                current = parent_map[current]
            return path[::-1]  # 返回反转的路径
        closed_list.add(current)
        # 检查周围的4个方向
        neighbors = [(current[0] + 1, current[1]), (current[0] - 1, current[1]),
                     (current[0], current[1] + 1), (current[0], current[1] - 1)]
        for neighbor in neighbors:
            if 0 <= neighbor[0] < map.shape[0] and 0 <= neighbor[1] < map.shape[1] and map[
                neighbor[0], neighbor[1]] == 0 and neighbor not in closed_list:
                if neighbor not in open_list:
                    open_list.append(neighbor)
                    parent_map[neighbor] = current
    return None  # 如果没有路径
# 计算路径
path = find_path(start, goal, map)
print("Path found:", path)
```

应用案例-服务机器人

- 应用场景：家居、餐厅、清洁、陪伴等
- 案例：如扫地机器人（例如iRobot Roomba）能够在家中自动清洁地板。它们配备了传感器和地图生成技术，能够自主导航，避开障碍物并完成清扫任务。
- 技术要求：自动导航、路径规划、环境感知、机器学习等。

```python
class Robot:
    def __init__(self, x, y, grid_size):
        self.x = x  # 当前位置的x坐标
        self.y = y  # 当前位置的y坐标
        self.grid_size = grid_size  # 网格大小
        self.direction = 'UP'  # 初始方向向上
    def move_forward(self):
        if self.direction == 'UP':
            self.y -= 1
        elif self.direction == 'DOWN':
            self.y += 1
        elif self.direction == 'LEFT':
            self.x -= 1
        elif self.direction == 'RIGHT':
            self.x += 1
    def detect_wall(self):
        # 简单的墙面检测，检查当前位置周围的障碍物
        # 这里假设网格中0表示空地，1表示墙面
        if self.x < 0 or self.x >= self.grid_size or self.y < 0 or self.y >= self.grid_size:
            return True
        return False
    def avoid_obstacles(self):
        if self.detect_wall():
            # 遇到墙面则改变方向
            self.direction = 'DOWN'  # 只是一个简单示例，实际应有更复杂的避障算法
    def navigate(self):
        while True:
            self.move_forward()  # 移动一步
            self.avoid_obstacles()  # 检测并避开障碍物
# 创建机器人并开始导航
robot = Robot(x=5, y=5, grid_size=10)
robot.navigate()
```

应用案例-无人机（Drone）

- 应用场景：农业监测、航拍、搜索与救援、物流运输等
- 案例：农业无人机可以自动喷洒农药，进行植物健康监测，极大提高农业生产效率。
- 技术要求：飞行控制、定位系统（GPS/IMU）、图像识别、路径规划等。

```python
class Drone:
    def __init__(self):
        self.altitude = 0  # 高度初始化为0
        self.speed = 0  # 速度初始化为0
    def take_off(self):
        self.altitude = 100  # 起飞到100米的高度
        self.speed = 20  # 设置飞行速度为20m/s
        print("Drone is taking off! Altitude:", self.altitude, "Speed:", self.speed)
    def fly(self):
        if self.altitude > 0:
            print("Drone is flying at", self.altitude, "meters with speed", self.speed, "m/s.")
        else:
            print("Drone has not taken off yet.")
    def land(self):
        self.altitude = 0  # 降落到地面
        self.speed = 0  # 停止飞行
        print("Drone has landed.")

# 使用无人机
drone = Drone()
drone.take_off()  # 起飞
drone.fly()  # 飞行
drone.land()  # 着陆
```

智能家居与物联网

智能家居与物联网

同济大学 TONGJI UNIVERSITY

智能家居

智能家居是指通过互联网将家中设备连接起来，并通过人工智能技术进行控制与管理，提供便捷、舒适、安全和节能的居住体验。用户可以通过智能设备如智能音响、智能门锁、智能灯光等来实现对家居环境的自动化管理和控制。

物联网（IoT）

物联网是指通过网络将物理设备与互联网连接，实现设备之间的信息交换和控制。在智能家居中，物联网将各个家庭设备（如传感器、摄像头、家电等）连接到一个智能平台，从而实现对家居设备的集中管理和自动控制。

应用技术

- **自然语言处理（NLP）：**用于语音助手（如Amazon Alexa、Google Assistant、Apple Siri）的开发，用户通过语音命令控制智能设备。例如，用户可以通过语音命令打开灯光、调节温度等。
- **机器学习：**机器学习帮助智能家居设备学习用户的行为模式，并进行个性化推荐和自动化控制。例如，智能温控系统（如Nest）通过学习用户的习惯和需求来自动调节家庭的温度，提供最佳的舒适度。
- **计算机视觉：**在智能安防中，计算机视觉技术用于监控摄像头的图像分析，能够识别进入家中的人物、物体，甚至检测异常活动。它还被广泛应用于人脸识别门锁和安全监控系统中。
- **传感器与自动化控制系统：**通过传感器监测家庭环境中的变化，如温度、湿度、光线、运动等，AI可以根据这些实时数据进行自动调节。例如，智能灯光系统可以根据环境亮度自动调整灯光强度，智能窗帘根据光照强度自动开启或关闭。
- **边缘计算等……**

应用案例-智能家居控制系统

- 应用描述：智能家居系统允许用户通过手机、语音助手或自动化规则来控制家中的灯光、温控、家电等设备。例如，通过手机应用调节家中的空调温度，或通过语音命令开关灯光。
- 技术组件：物联网设备（传感器、智能家居硬件）、智能语音助手（如Alexa、Google Assistant）、云平台。

```python
import time
# 假设有智能设备类用于控制家电
class SmartDevice:
    def __init__(self, device_name):
        self.device_name = device_name
        self.status = "off"
    def turn_on(self):
        self.status = "on"
        print(f"{self.device_name} is turned ON.")
    def turn_off(self):
        self.status = "off"
        print(f"{self.device_name} is turned OFF.")
# 智能家居控制系统
class SmartHomeSystem:
    def __init__(self):
        self.devices = {
            "light": SmartDevice("Light"),
            "air_conditioner": SmartDevice("Air Conditioner")
        }
    def control_device(self, device_name, action):
        device = self.devices.get(device_name)
        if device:
            if action == "on":
                device.turn_on()
            elif action == "off":
                device.turn_off()
            else:
                print("Invalid action")
        else:
            print(f"{device_name} not found in the system.")
# 测试智能家居控制
smart_home = SmartHomeSystem()
# 控制灯光和空调
smart_home.control_device( device_name: "light",  action: "on")
smart_home.control_device( device_name: "air_conditioner",  action: "off")
```

应用案例-智能安防系统

- 应用描述：智能安防系统通过摄像头、传感器、门锁等设备实时监控家庭安全。通过智能手机，用户可以随时查看家中情况，并进行远程操作，如锁门、关闭窗帘、打开报警器等。
- 技术组件：传感器（如门窗传感器、摄像头）、云平台、智能报警系统、机器学习算法（人脸识别等）。

```python
import random
# 假设有智能安防设备类用于控制摄像头和传感器
class SmartSecurityDevice:
    def __init__(self, device_name):
        self.device_name = device_name
        self.status = "off"
    def turn_on(self):
        self.status = "on"
        print(f"{self.device_name} is turned ON.")

    def turn_off(self):
        self.status = "off"
        print(f"{self.device_name} is turned OFF.")

    def detect_motion(self):
        # 模拟传感器检测到运动
        return random.choice([True, False])
# 智能安防系统
class SmartSecuritySystem:
    def __init__(self):
        self.devices = {
            "door_sensor": SmartSecurityDevice("Door Sensor"),
            "camera": SmartSecurityDevice("Camera")
        }
    def activate_system(self):
        print("Activating security system...")
        for device in self.devices.values():
            device.turn_on()
    def deactivate_system(self):
        print("Deactivating security system...")
        for device in self.devices.values():
            device.turn_off()
    def check_for_intruders(self):
        if self.devices["door_sensor"].detect_motion():
            print("Motion detected! Alerting user!")
            self.devices["camera"].turn_on()
```

应用案例-智能健康监测

- 应用描述：通过连接健康监测设备（如智能手环、智能体重秤等），用户可以实时获取自己的健康数据（如心率、体温、血糖等）。这些数据通过物联网设备上传至云平台，医生可以远程监控病人的健康状况。
- 技术组件：传感器、智能穿戴设备、云平台、大数据分析、AI算法。

```python
import random

# 假设有智能健康设备类
class HealthMonitoringDevice:
    def __init__(self, device_name):
        self.device_name = device_name

    def get_heart_rate(self):
        # 模拟获取心率数据
        return random.randint( a: 60,  b: 100)

# 智能健康监测系统
class HealthMonitoringSystem:
    def __init__(self):
        self.device = HealthMonitoringDevice("Heart Rate Monitor")

    def monitor_health(self):
        heart_rate = self.device.get_heart_rate()
        print(f"Current heart rate: {heart_rate} BPM.")

        # 假设如果心率超过90，发送警报
        if heart_rate > 90:
            print("Warning: High heart rate detected! Consider taking a break.")

# 测试健康监测系统
health_monitor = HealthMonitoringSystem()
health_monitor.monitor_health()
```

金融行业

AI在金融行业的应用

- **风险管理与信用评估**

AI通过分析客户的信用历史、交易行为和其他相关数据，提供精准的信用评分，并帮助银行做出更加科学的贷款决策。此外，AI还可以预测潜在的信用风险，帮助金融机构管理贷款风险。

- **欺诈检测与预防**

AI可以通过实时监控交易，使用异常检测算法识别潜在的欺诈行为。例如，机器学习模型能够识别出与正常交易模式不同的可疑交易，及时触发警报，并采取相应的预防措施。

- **智能投顾（Robo-advisors）**

AI驱动的智能投顾为个人投资者提供定制化的投资建议。通过分析投资者的偏好、风险承受能力及市场趋势，AI能够自动推荐最佳的投资组合，并进行定期调整。

- **自动化交易**

高频交易系统利用AI进行市场数据分析和算法交易，AI模型可以在毫秒之间完成交易决策，执行快速的股票买卖操作，以获取市场上的微小利润。

金融行业

同济大学 TONGJI UNIVERSITY

应用技术

- **机器学习：**机器学习是AI在金融行业中的基础技术之一。它通过训练算法从大量的历史数据中学习模式和趋势，从而做出预测和决策。比如，机器学习可以预测股市波动、评估客户信用、检测欺诈行为等。
- **自然语言处理（NLP）：**NLP技术使得计算机能够理解和生成自然语言，这在金融行业中主要用于客服聊天机器人、情感分析、文本分析（如财经新闻和报告的自动解析）等。NLP能够帮助银行和金融机构更好地理解客户需求并提供个性化服务。
- **大数据分析：**金融行业涉及大量的交易数据、客户行为数据等。AI可以通过大数据技术对这些海量数据进行实时处理和分析，为决策者提供精准的业务洞察，如客户偏好、市场趋势等。
- **智能投顾（Robo-advisors）：**智能投顾通过算法为投资者提供定制化的投资建议。基于用户的风险偏好、投资目标和历史数据，AI模型能够设计个性化的投资组合，并在市场变化时做出调整。
- **深度学习：**深度学习在金融行业中的应用主要体现在算法交易、市场预测和高频交易等领域。深度学习模型能够识别复杂的数据模式并进行精确预测。

金融行业

同济大学 TONGJI UNIVERSITY

应用案例-信贷评分（Credit Scoring）

- 应用场景：金融机构使用信贷评分模型来评估个人或企业的信用风险，决定是否发放贷款以及贷款额度。信贷评分通常基于个人的历史信用记录、收入、债务和其他相关特征。
- 技术应用：机器学习模型（如逻辑回归、决策树、随机森林）可以用于预测借款人是否会违约，从而做出信用评估。

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# 加载信用评分数据集
data = pd.read_csv('credit_score_data.csv')

# 数据预处理（假设数据中包含‘信用评分’, ‘收入’, ‘债务’等列）
X = data[['收入', '债务', '历史信用记录']]  # 特征
y = data['信用违约']  # 标签

# 将数据集分为训练集和测试集
X_train, X_test, y_train, y_test = train_test_split( *arrays: X, y, test_size=0.2, random_state=42)

# 使用随机森林模型进行训练
model = RandomForestClassifier()
model.fit(X_train, y_train)

# 进行预测
y_pred = model.predict(X_test)

# 评估模型准确性
accuracy = accuracy_score(y_test, y_pred)
print(f'模型准确率: {accuracy:.2f}')
```

金融行业

同济大学 TONGJI UNIVERSITY

应用案例-股票价格预测

- 应用场景：投资者和机构使用机器学习和深度学习技术预测股票市场的未来走势，帮助做出买卖决策。通过分析历史价格数据、市场新闻、技术指标等因素，模型可以预测未来的股票价格。
- 技术应用：时间序列分析（如ARIMA模型）、回归分析、深度学习（如LSTM）等都可以用于股票价格预测。

```python
import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
# 加载股票数据
stock_data = pd.read_csv('stock_prices.csv')
# 数据预处理（假设数据中包含“收盘价”列）
close_prices = stock_data['Close'].values
close_prices = close_prices.reshape(-1, 1)
# 归一化数据
scaler = MinMaxScaler(feature_range=(0, 1))
scaled_prices = scaler.fit_transform(close_prices)
# 构建数据集（使用过去60天的数据预测下一天的价格）
X, y = [], []
for i in range(60, len(scaled_prices)):
    X.append(scaled_prices[i-60:i, 0])
    y.append(scaled_prices[i, 0])
X, y = np.array(X), np.array(y)
# 将X调整为LSTM的输入形状
X = np.reshape(X, newshape: (X.shape[0], X.shape[1], 1))
# 创建LSTM模型
model = Sequential()
model.add(LSTM(units=50, return_sequences=True, input_shape=(X.shape[1], 1)))
model.add(LSTM(units=50, return_sequences=False))
model.add(Dense(units=1))  # 输出预测的价格
# 编译模型
model.compile(optimizer='adam', loss='mean_squared_error')
# 训练模型
model.fit(X, y, epochs=10, batch_size=32)
# 预测未来股票价格
predicted_price = model.predict(X[-1].reshape(1, 60, 1))
predicted_price = scaler.inverse_transform(predicted_price)
print(f'预测的未来股票价格: {predicted_price[0][0]:.2f}')
```

金融行业

同济大学 TONGJI UNIVERSITY

应用案例-欺诈检测（Fraud Detection）

- 应用场景：银行和金融机构利用机器学习模型检测欺诈交易，如信用卡盗刷、虚假交易等。通过分析用户的交易行为和模式，模型可以识别出异常交易，并进行警报。
- 技术应用：监督学习模型（如随机森林、SVM）和无监督学习（如异常检测）广泛应用于欺诈检测。

```python
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

# 加载交易数据集
data = pd.read_csv('transactions.csv')

# 数据预处理（假设数据中包含“交易金额”，“交易频率”等特征）
X = data[['交易金额', '交易频率', '账户活动']]  # 特征
y = data['是否欺诈']  # 标签

# 将数据集分为训练集和测试集
X_train, X_test, y_train, y_test = train_test_split( *arrays: X, y, test_size=0.2, random_state=42)

# 使用随机森林模型进行训练
model = RandomForestClassifier()
model.fit(X_train, y_train)

# 进行预测
y_pred = model.predict(X_test)

# 打印分类报告
print(classification_report(y_test, y_pred))
```

医疗健康

AI在医疗健康中的应用场景

- **疾病诊断：**AI在疾病诊断中的应用可以通过自动化分析医学影像和历史病例，帮助医生发现潜在的健康问题。例如，AI可以在X光片、CT、MRI影像中识别肿瘤、结节等病变。
- **个性化治疗：**AI技术能够分析大量患者的历史数据，包括基因组学、疾病背景、生活习惯等信息，从而为每位患者定制个性化的治疗方案。例如，AI可以根据患者的基因信息预测药物反应，帮助选择最佳药物和剂量。
- **药物研发：**AI加速了新药的研发过程，通过对大量的生物医学数据和化学结构进行分析，AI能够识别潜在的药物分子，并加速临床试验设计和药物筛选。
- **智能辅助系统：**AI提供的辅助决策支持系统能够帮助医生提高诊疗效率、减少人为错误。例如，AI辅助系统能够实时分析病人的健康数据，提出诊断和治疗建议。
- **机器人手术与康复：**AI驱动的机器人能够执行复杂的手术，并通过实时数据分析，提高手术的精度与成功率。此外，康复机器人可以帮助病人恢复功能，提供精准的康复训练。

医疗健康

同济大学 TONGJI UNIVERSITY

应用技术

- **机器学习（ML）**：通过分析大量医疗数据，机器学习模型能够识别疾病的早期迹象并进行诊断预测。例如，通过分析影像数据或基因数据，帮助早期发现癌症、心脏病等。
  应用实例：疾病预测、诊断和个性化治疗方案设计。
- **深度学习（DL）**：深度学习通过多层神经网络识别复杂模式，特别适合医疗影像分析，如X光、CT、MRI等，自动识别病变区域，辅助诊断。
  应用实例：癌症、脑部疾病、肺炎等医学影像分析。
- **自然语言处理（NLP）**：NLP使计算机能够理解并分析医疗文献、病历和患者报告，帮助医生提取关键信息，加速诊疗决策。
  应用实例：自动解读病历、智能问诊、支持治疗方案制定。
- **计算机视觉（CV）**：计算机视觉用于自动化医疗影像分析，识别病变或异常区域，广泛应用于眼科、放射科等领域。
  应用实例：皮肤病变、肺部结节、乳腺癌检测。
- **机器人技术**：AI驱动的手术机器人可以减少操作误差，提高精度，缩短患者恢复时间。
  应用实例：达芬奇手术机器人进行微创手术。

医疗健康

同济大学 TONGJI UNIVERSITY

应用案例-疾病预测与诊断

- 应用：利用机器学习算法，分析患者的健康数据（如病历记录、体检结果等），预测疾病的发生。常见的应用包括癌症预测（如乳腺癌、肺癌等）和心脏病预测等。
- 例子：通过分析X光片、CT图像、MRI图像等，AI可以帮助医生识别潜在的病变区域，或者预测病情的进展。

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# 1. 加载患者健康数据
data = pd.read_csv('heart_disease_data.csv')

# 2. 数据预处理：处理缺失值、标准化等
X = data.drop( labels: 'target', axis=1)  # 特征
y = data['target']  # 标签（是否患病）

# 3. 切分数据集
X_train, X_test, y_train, y_test = train_test_split( *arrays: X, y, test_size=0.2, random_state=42)

# 4. 创建并训练模型
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# 5. 预测测试集
y_pred = model.predict(X_test)

# 6. 评估模型
accuracy = accuracy_score(y_test, y_pred)
print(f'Model Accuracy: {accuracy * 100:.2f}%')
```

应用案例-医学影像分析

- 应用：AI用于医学影像（如CT、MRI、X光等）的自动分析，帮助医生更高效地发现异常，提取诊断信息。可以用于肿瘤检测、骨折检测等任务。
- 例子：使用卷积神经网络（CNN）自动分析胸部X光片，检测肺部结节或肺炎。

```python
import tensorflow as tf
from tensorflow.keras.preprocessing import image
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.applications.resnet50 import preprocess_input, decode_predictions

# 1. 加载预训练的卷积神经网络（CNN）模型
model = ResNet50(weights='imagenet')

# 2. 加载并预处理X光片图像
img_path = 'chest_xray.jpg'
img = image.load_img(img_path, target_size=(224, 224))
img_array = image.img_to_array(img)
img_array = tf.expand_dims(img_array, axis=0)  # 扩展维度，适应模型输入
img_array = preprocess_input(img_array)

# 3. 进行预测
predictions = model.predict(img_array)

# 4. 解码预测结果
decoded_predictions = decode_predictions(predictions, top=3)[0]
for i, (imagenet_id, label, score) in enumerate(decoded_predictions):
    print(f"{i+1}: {label} ({score:.2f})")
```

应用案例–个性化治疗与药物推荐

- 应用：基于患者的基因信息、病史等数据，AI能够为患者提供个性化的治疗方案或药物推荐，帮助提升治疗效果。
- 例子：通过对患者的基因组数据分析，AI可以推荐适合的药物，避免传统药物使用的副作用。

```python
import pandas as pd
from sklearn.neighbors import NearestNeighbors

# 1. 加载患者病历数据（包括症状、诊断等）
data = pd.read_csv('patient_medical_records.csv')

# 2. 假设每个患者有一组症状特征
X = data.drop( labels: 'patient_id', axis=1)  # 特征：症状等
y = data['patient_id']  # 标签：患者ID，假设药物推荐基于症状相似性

# 3. 使用最近邻算法找到症状相似的患者
model = NearestNeighbors(n_neighbors=5, algorithm='ball_tree')
model.fit(X)

# 4. 输入新患者的症状数据
new_patient = [[1, 0, 0, 1, 0]]  # 假设新患者的症状特征向量

# 5. 获取与新患者相似的患者
distances, indices = model.kneighbors(new_patient)

# 6. 基于相似患者的药物推荐
recommended_drugs = []
for idx in indices[0]:
    recommended_drugs.append(data.iloc[idx]['recommended_drug'])

print(f"Recommended drugs for the new patient: {recommended_drugs}")
```